

Chapter 1

OOP Part B — Static, Inheritance, Polymorphism

ENEL712 Embedded Systems Design — Lecture 16

Summary

OOP continued: static members and constants, inheritance and the base keyword, polymorphism via overloading, enumerations, and a coaxial-cable modelling exercise.

Topics Covered

- Static methods and data members
- Inheritance
- Polymorphism and overloading
- Enumerations
- Coaxial cable programming exercise

1.1 Static Methods and Data Members

- Static members belong to the class itself, not to instances.
- Static methods: utility functions called as `Class.Method()`; can only call other static members.
- Static data members: shared fields, e.g., `public static int numCircles = 0;`.
- Constants: `const` creates a static, immutable field (e.g., `Math.PI`).
- `Math.Sqrt()` is a classic example — only depends on the argument.

1.2 Inheritance

- Promotes code reuse — derived class inherits from a base class.
- Syntax: `class DerivedClass : BaseClass.`
- A class can derive from only one base class.

- Good practice: derived constructor explicitly calls the base constructor with base.
- Example: Employee base class with pay rate; Manager and Cleaner derived classes.

1.3 Polymorphism and Overloading

- Polymorphism = many forms; here implemented via overloading.
- Function overloading: multiple methods with the same name but different parameter types/counts.
- Operator overloading: redefine $+$ or $-$ for custom classes; methods must be public static.
- Constraints: cannot change precedence, invent new operators, or change behaviour for built-in types.

1.4 Enumerations (enum)

- Assign meaningful names to fixed numeric values for readability — common in switch-case.
- Example: enum Season {Spring, Summer, Autumn, Winter}; — Autumn corresponds to 2.

1.5 Coaxial Cable Class Exercise

Model a coaxial cable with two private radius members (inner a, outer b) and public methods to compute physical properties.

Parameter	Formula	Physical meaning
Series Inductance L	$L = (\mu_0 \mu_r \ell / 2\pi) \cdot \ln(b/a)$	Opposition to changes in current; depends on geometry and magnetic properties.
Shunt Capacitance C	$C = (2\pi \varepsilon_0 \varepsilon_r \ell) / \ln(b/a)$	Ability to store charge; depends on geometry and dielectric.

- μ_0 : permeability of free space.
- μ_r : relative permeability (dimensionless, vs. μ_0).
- ε_0 : permittivity of free space.
- ε_r : relative permittivity (dielectric constant).

1.6 Use Cases & Applications

- static class for utilities — Math, File, Console, and your own Helpers — when there's no per-instance state.
- Inheritance for shared behaviour: an Employee base with PayEmployee() and Manager / Cleaner specialisations.
- const for compile-time constants like Math.PI, conversion factors, or sensor part numbers.
- Operator overloading for math types (Vector2D, Complex) so client code stays readable: $a + b$ instead of Add(a, b).
- enum for finite, named state sets — much safer than 'magic numbers' inside switch statements.

1.7 Worked Example: Employee Hierarchy

```
public class Employee
{
    public string Name { get; }
    public decimal HourlyRate { get; }

    public Employee(string name, decimal rate)
    { Name = name; HourlyRate = rate; }

    public virtual decimal WeeklyPay(int hours) => HourlyRate * hours;
}

public class Manager : Employee
{
    public decimal Bonus { get; }
    public Manager(string n, decimal r, decimal b) : base(n, r) { Bonus = b; }
    public override decimal WeeklyPay(int h) => base.WeeklyPay(h) + Bonus;
}
```

1.8 Worked Example: Coaxial Cable Model

Translate the coaxial cable formulas into a class with private radii and public computed properties.

```
public class CoaxCable
{
    private readonly double a, b, length;
    private const double mu0 = 4e-7 * Math.PI;
    private const double eps0 = 8.854e-12;
    public double MuR { get; set; } = 1.0;
    public double EpsR { get; set; } = 2.25; // typical PE dielectric

    public CoaxCable(double inner, double outer, double l)
    { a = inner; b = outer; length = l; }

    public double SeriesInductance =>
        (mu0 * MuR * length / (2 * Math.PI)) * Math.Log(b / a);

    public double ShuntCapacitance =>
        (2 * Math.PI * eps0 * EpsR * length) / Math.Log(b / a);
}
```

1.9 Operator Overloading Sketch

```
public struct Vector2D
{
    public double X, Y;
    public Vector2D(double x, double y) { X = x; Y = y; }

    public static Vector2D operator +(Vector2D a, Vector2D b) =>
        new Vector2D(a.X + b.X, a.Y + b.Y);

    public static Vector2D operator *(Vector2D a, double k) =>
        new Vector2D(a.X * k, a.Y * k);
}
```

Key Definitions

Static Member

Member belonging to the class itself, not instances.

Static Method

Utility function that uses only static members.

Static Data Member

Field shared by all instances of a class.

Inheritance

Mechanism letting a derived class acquire base-class members.

Base Class

Class providing common features to derived classes.

Derived Class

Class that inherits from a base class and may extend it.

Function Overloading

Same name, different parameter list.

Operator Overloading

Redefining an operator for custom classes (public static methods).

Enum

Set of named integer constants improving readability.

Single Inheritance

C#'s rule that a class can have at most one direct base class (interfaces are the way to mix multiple contracts).

base Keyword

Used in a derived class to call the base constructor or invoke the base implementation of an overridden method.

readonly

Field modifier making the value settable only in declaration or constructor — a runtime equivalent of const.

Magic Number

Unnamed numeric literal embedded in code; usually replaced with a const or enum for clarity.

Sources

- Lecture 16.pdf